# FIT3155: Solutions to conceptual questions in Week 3 Lab sheet

1. Consider the following pattern string drawn from the alphabet $\Sigma = \{\mathtt{x}, \mathtt{y}, \mathtt{z}\}$:

| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
|---|---|---|---|---|---|---|---|---|---|---|
| y | z | z | y | z | x | y | z | z | y | z |

   Work out (on paper) (i) standard bad-character shift array and (ii) the extended bad-character shift table for the above pattern.

   **Ans. 1**

   (i) Standard bad-character shift array:

| | x | y | z |
|---|---|---|---|
| $R[\cdot]$ | 6 | 10 | 11 |

   (ii) Extended bad-character shift table:

| | x | y | z |
|---|---|---|---|
| $R[1][\cdot]$ | 0 | 0 | 0 |
| $R[2][\cdot]$ | 0 | 1 | 0 |
| $R[3][\cdot]$ | 0 | 1 | 2 |
| $R[4][\cdot]$ | 0 | 1 | 3 |
| $R[5][\cdot]$ | 0 | 4 | 3 |
| $R[6][\cdot]$ | 0 | 4 | 5 |
| $R[7][\cdot]$ | 6 | 4 | 5 |
| $R[8][\cdot]$ | 6 | 7 | 5 |
| $R[9][\cdot]$ | 6 | 7 | 8 |
| $R[10][\cdot]$ | 6 | 7 | 9 |
| $R[11][\cdot]$ | 6 | 10 | 9 |

2. Determine the worst-case time and space complexities to **construct**:

   - the standard bad-character shift array, and
   - the extended bad-character shift table.

   Separately, after construction, what is the worst-case time to **retrieve** any shift value from each of the above?

   **Ans. 2**

   *(i) Standard bad-character shift array*:

   For the pattern pat$[1 \dots m]$, preprocessing creates a 1D array data structure $R[1 \dots |\Sigma|]$, where each $R[c]$ of the array stores the position of the rightmost occurrence of a character $c \in \Sigma$ observed in the pattern pat$[1 \dots m]$:

```
1 Initialize R[1...|Σ|] ← {0, 0, ..., 0}
2 looping for each character c ∈ pat[1...m]
3   assign R[c] ← i
```

   From this preprocessing logic, it remains clear that the space requirement is $O(|\Sigma|)$, with $O(|\Sigma| + m)$ effort to fill the 1D array.

   *(ii) Extended bad-character shift table*:

   For the pattern pat$[1 \dots m]$, we want to preprocess now a 2D $m \times |\Sigma|$ matrix/table data structure ($R[1 \dots m][1 \dots |\Sigma|]$), where each $R[k][c]$ stores the position of the rightmost occurrence of the character $c \in \Sigma$ observed in the pattern to the left of ($<$) position $k$:

```
1  #Step 1: Initialize 2D table (1-based indexing)
2  R[1...m][1...|Σ|] ← {{0, 0, ..., 0}, {0, 0, ..., 0}, ... {0, 0, ..., 0}}
3
4  #Step 2: Loop over pattern characters while seeding
       the 2D table
5  looping k from 1...m − 1
6    c = pat[k]
7    assign R[k + 1][c] ← k
8
9  #Step 3: Propagate the seeded cell values in each
       column to make all table columns monotonic
10 looping for all c ∈ Σ
11   looping k from 2 to m
12     when R[k][c] = 0, assign R[k][c] ← R[k − 1][c]
```

   Auxiliary space requirement is mainly to store the $m \times \Sigma$ table. Step 1 requires $m \times \Sigma$ effort to initialize the table. Step 2 requires one pass over the pattern, with constant effort within that loop. Step 3 is tantamount to checking all cells of the 2D table, requiring $O(m \times \Sigma)$ effort. Hence, space and time complexity is bounded above by $O(m \times \Sigma)$.

3. For a pattern of length $m$, can a data structure supporting the **extended** bad-character shift function (beyond the one that was realized as an $m \times |\Sigma|$ table as we saw in the lecture) be constructed in $O(m+|\Sigma|)$ worst-case time and space?

   **If Yes:** How? And what is the worst-case access time to **retrieve** shift values from this constuction?

   **If No:** Why not?

   **Ans. 3**

   Yes.

   Another way to preprocess the extended bad-character shift function is using a list of variable-length lists, one per character in $\Sigma$. Step 1 creates $|\Sigma|$ empty lists. Step 2 scans the pattern once, appending each character's position $k$ to its corresponding list (which by construction, for each character in the alphabet, results in a list of positions stored in a sorted order).

```
1 # Step 1: Instantiate empty lists for each
     character in the alphabet
2 Instatiate R[1...|Σ|][∅]
3
4 # Step 2: Scan pattern and append positions of
     each observed character to its list.
5 for k = 1 to m
6   append k to the list R[pat[k]][...]
```

   Space required is $O(|\Sigma|+m)$ since there are $|\Sigma|$ lists containing exactly $m$ total entries after the construction. Construction time is also $O(|\Sigma|+m)$ over the two successive steps.

   To find the rightmost position that is to the left of any $k$ for any possible character $c$ in the alphabet, a retrieval function can be written that relies on binary searching the list $R[c]$, requiring $O(\log m)$ effort in the worst-case (as the maximum possible length of any list is $m$).

4. Consider the following pattern string drawn from the alphabet $\Sigma = \{\texttt{a}, \texttt{b}, \texttt{c}\}$:

| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| a | b | a | a | a | b | a | c | b | a | a | b | a | a | a | b |

Construct for the above pattern by hand

(a) its good-suffix shift array, and

(b) its matched-prefix shift array.

**Ans. 4**

| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| a | b | a | a | a | b | a | c | b | a | a | b | a | a | a | b | |
| **Good Suffix Array** | | | | | | | | | | | | | | | | |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 6 | 0 | 0 | 12 | 2 | 9 | 15 |
| **Matched Prefix Array** | | | | | | | | | | | | | | | | |
| 16 | 6 | 6 | 6 | 6 | 6 | 6 | 6 | 6 | 6 | 6 | 2 | 2 | 2 | 2 | 0 | 0 |

*Note:* When utilizing the **good suffix** (`gs`) and **matched prefix** (`mp`) arrays, the values at index 1 (in 1-based indexing) are never accessed. Consequently, the specific values stored at `gs`[1] and `mp`[1] are of no consequence to the algorithm's execution.

5. Consider the following **periodic** text and pattern over the alphabet $\Sigma = \{\mathtt{a}, \mathtt{b}, \mathtt{c}, \mathtt{d}, \mathtt{e}\}$:

```
       1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40
txt = a b c d e a b c d e a b c d e a b c d e a b c d e a b c d e a b c d e a b c d e

       1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
pat = a b c d e a b c d e a b c d e
```

Using the mechanics of Boyer-Moore algorithm applicable for this scenario, work out how many explicit character comparisons are performed to find all occurrences of the pattern in the text?

**Ans. 5**

Iteration 1: The region txt$[1 \ldots 15]$ is opposing pat$[1 \ldots 15]$. Scanning right to left pat$[\overleftarrow{1 \ldots 15}]$ matches txt$[\overleftarrow{1 \ldots 15}]$. Therefore, 15 explicit comparisons for this iteration. Shift pat right by $m-$matchedPrefix$[2] = 15 - 10 = 5$. Note, matchedPrefix$[2] = 10$ because the longest suffix of the string pat$[2 \ldots m]$ (or, in other words, the longest **proper** suffix* of the whole pattern) that matches the pattern's prefix is of length 10, pat$[6 \ldots 15]$ = pat$[1 \ldots 10]$.

Iterations 2: The region txt$[6 \ldots 20]$ is opposing pat$[1 \ldots 15]$. Scanning right to left pat$[\overleftarrow{11 \ldots 15}]$ matches txt$[\overleftarrow{16 \ldots 20}]$. So, only 5 explicit comparisons. No further comparisons necessary beyond this because, applying the optimization/relationship implicit from previous iteration (which formed the basis of the shift in iteration 1), pat$[1 \ldots 10]$ has to be same as txt$[6 \ldots 15]$. After this, shift pat right by $m -$ matchedPrefix$[2] = 15 - 10 = 5$.

Iterations 3: The region txt$[11 \ldots 25]$ is opposing pat$[1 \ldots 15]$. Scanning right to left pat$[\overleftarrow{11 \ldots 15}]$ matches txt$[\overleftarrow{21 \ldots 25}]$. So, only 5 explicit comparisons. No further scans necessary as, applying the optimization/relationship implicit from previous iteration (which formed the basis of the shift in iteration 2), pat$[1 \ldots 10]$ has to be same as txt$[11 \ldots 20]$. Shift pat right by $m -$ matchedPrefix$[2] = 15 - 10 = 5$.

$\vdots$ ditto for remaining iterations 4, 5, and 6.

Total number of explicit comparisons: $15 + 5 + 5 + 5 + 5 + 5 = 40$.

*A **proper** suffix of a string is any suffix of the string that is not the string itself.

6. Consider some iteration of the Boyer-Moore algorithm involving the pattern $\texttt{pat}[1 \ldots m]$ and the text $\texttt{txt}[1 \ldots n]$. Suppose that in this iteration the region $\texttt{txt}[j \ldots j+m-1]$ of the text is being compared with pat.

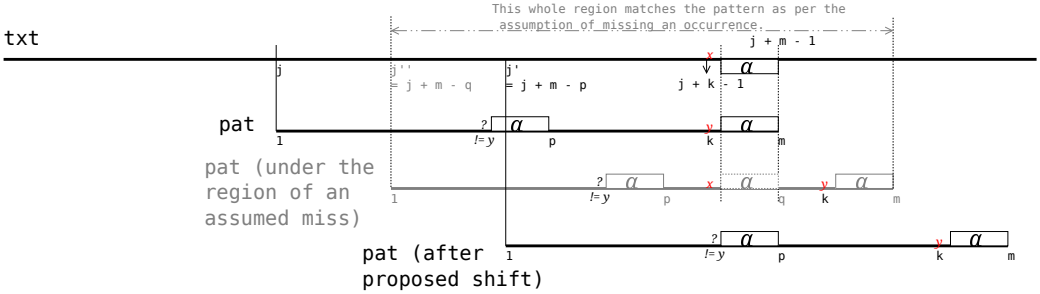
   Prove that if a right-to-left scan results in a mismatch at position $k \leq m$ and the value in the good-suffix array satisfies $\texttt{gs}[k+1] > 0$ (using 1-based indexing note), then the shift by $m - \texttt{gs}[k+1]$ using the good suffix rule is **safe**, i.e., such a shift can never move pat past a valid occurrence in txt.

**Ans. 6**

Consider an iteration where $\text{pat}[1 \ldots m]$ is being compared with a region of text, $\text{txt}[j \ldots j+m-1]$, and right-to-left scan in that region yielded a mismatch at position $k$ in the pattern ($\text{pat}[k] \neq \text{txt}[j+k-1]$) and a good suffix $\alpha = \text{pat}[k+1 \ldots m] = \text{txt}[j+k \ldots j+m-1]$.

By definition, if $\texttt{gs}[k+1] = p > 0$ (1 based indexing), then $p$ has to be the end-point of the rightmost (other) instance of the good suffix $\alpha$ in the pattern such that the character preceding that instance, $\text{pat}[p - |\alpha|] \neq \text{pat}[k]$).

The proposed good suffix shift rule of Boyer-Moore algorithm requires us to shift the pattern rightwards by $m - p$ positions (so that the instance ending at $p$ is now opposing an $\alpha$ region in the text ending at $j+m-1$. This means that, after this proposed shift, the whole pattern will be aligned under the region $\text{txt}[j' \ldots j'+m-1]$, where $j' = j+m-p$. †.




We will prove that this shift is indeed safe using a proof by contradiction.

For this, first assert/assume that shifting the pattern from $j \to j'$ is **unsafe** and it skips over an occurrence of the pattern in the text starting at some $j'' = j+m-q < j'$. In other words, the assertion is saying we should have shifted rightwards the pattern more conservatively by only (some) $m-q$ positions, instead of the proposed $m-p$ positions suggested by the good suffix shift rule. From this assertion (of missing an instance), we are assuming:

$$\text{txt}[j'' \ldots j''+m-1] = \text{pat}[1 \ldots m] \qquad \text{and} \qquad j < j'' < j'.$$

Shifting by $m-q$ positions implies $\text{pat}[q]$ is opposite $\text{txt}[j+m-1]$. So,if shifting from $j \to j''$ yielded a full match, then the characters $\text{pat}[q-|\alpha|+1 \ldots q]$ will be opposite $\text{txt}[j+k \ldots j+m-1]$, that is the region which previously matched with the good suffix $\alpha$. Why? We asserted the whole region matches after the shift from $j \to j''$ (which the good suffix shift rule missed). If $\text{txt}[j'' \ldots j''+m-1] = \text{pat}[1 \ldots m]$ we will have to conclude that the corresponding partial regions are equivalent:

$$\text{pat}[q-|\alpha|+1 \ldots q] = \text{txt}[j+k \ldots j+m-1] = \alpha.$$

Also, if the whole region starting $j''$ matches the pattern, then the character $\text{pat}[q-|\alpha|]$ preceding the instance of $\alpha$ ending at $q$ has to be the same as the previous iteration's 'bad-character' $\text{txt}[j+k-1]$, and thereby different from $\text{pat}[k]$:

$$\text{pat}[q-|\alpha|] = \text{txt}[j+k-1] \neq \text{pat}[k].$$

However, from our assertion

$$j'' < j' \implies j+m-q < j+m-p \implies q > p.$$

This means there is a another instance of $\alpha$ ending at position $q > p$ in the pattern whose preceding character is not same as $\text{pat}[k]$, leading to a contradiction of the very definition of $\texttt{gs}[k+1] = p$. If such an instance existed, $\texttt{gs}[k+1] = q$, and not $p$.

Therefore the assertion that the good suffix shift could skip unsafely over an occurrence of the pattern in text is invalid, and proposed goodsuffix shift from $j \to j'$ has to yield a safe shift.

†'j(u)mp' – pun intended! ☺

7. Consider some iteration of the Boyer-Moore algorithm involving the pattern $\texttt{pat}[1 \ldots m]$ and the text $\texttt{txt}[1 \ldots n]$. Suppose that in this iteration the region $\texttt{txt}[j \ldots j+m-1]$ of the text is being compared with pat.

   Prove that if a right-to-left scan results in a mismatch at position $k \leq m$ and the value in the good-suffix array satisfies $\texttt{gs}[k+1] = 0$ (using 1-based indexing note), then the shift by $m - \texttt{mp}[k+1]$ using the matched prefix rule is **safe**, i.e., such a shift can never move pat past a valid occurrence in txt.

**Ans. 7**

Consider an iteration where $\text{pat}[1 \ldots m]$ is being compared with the region of text, $\text{txt}[j \ldots j+m-1]$, and right-to-left scan in that region yielded a mismatch at position $k$ in the pattern ($\text{pat}[k] \neq \text{txt}[j+k-1]$) and the good suffix as $\alpha = \text{pat}[k+1 \ldots m]$.

By definition, if $\texttt{gs}[k+1] = 0$ (in 1 based indexing), this means there exists no other instance of the good suffix $\alpha$ in the pattern such that (the character preceding that instance) $\text{pat}[p-|\alpha|] \neq \text{pat}[k]$.

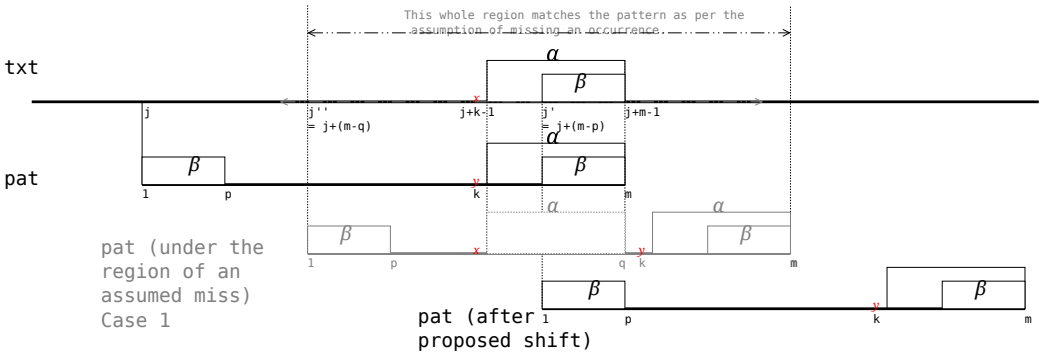
The proposed suffix shift rule of the Boyer-Moore algorithm requires us to shift using the matched prefix rule: If $\beta$ is the longest suffix of $\text{pat}[k+1 \ldots m]$ that matches the prefix $\text{pat}[1 \ldots p = |\beta|]$, then shift the pattern rightwards by $m-p$ positions (so that the matched prefix lines up opposing the $\beta$ region in the text, $\text{txt}[j+m-p \ldots j+m-1]$). This means that after this proposed shift, the pattern will be aligned opposite the region $\text{txt}[j' \ldots j'+m-1]$, where $j' = j+m-p$.

We will prove that this shift is indeed safe using a proof by contradiction. Assert that shifting the pattern from $j \to j'$ is unsafe and skips over an occurrence of the pattern in the text starting at some $j'' = j+m-q < j'$.

In other words, the assertion in this case is saying we should have shifted the pattern more conservatively, by $m-q$ positions rightwards, instead of the $m-p$ positions being suggested by the matched prefix shift rule. Thus, from this assertion, we have:

$$\text{txt}[j'' \ldots j''+m-1] = \text{pat}[1 \ldots m] \qquad \text{and} \qquad j < j'' < j'.$$

**Case 1:** Consider the case where $q \geq |\alpha|$:




If shifting from $j \to j''$ yielded a full match and $q \geq |\alpha|$, then

$$\text{pat}[q-|\alpha|+1 \ldots q] = \text{txt}[j+k \ldots j+m-1] = \alpha.$$

Why? We asserted that, after the shift from $j \to j''$, the whole regions matched: $\text{txt}[j'' \ldots j''+m-1] = \text{pat}[1 \ldots m]$. From this, we will have to conclude that

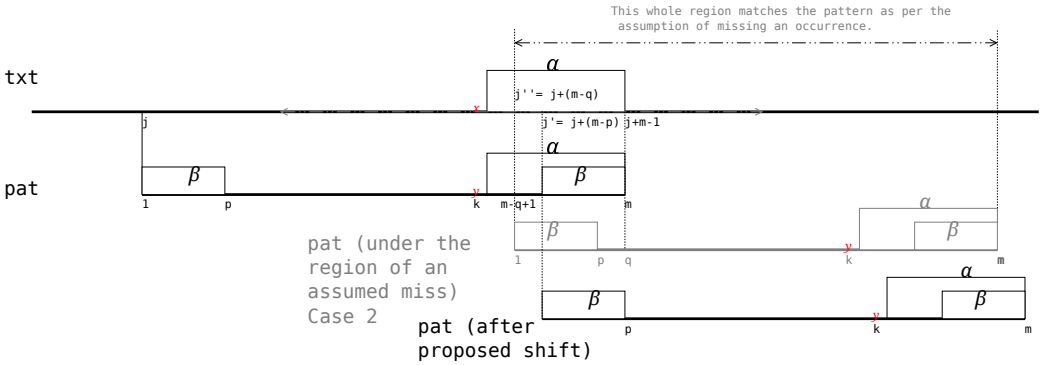
$$\text{pat}[q-|\alpha|+1 \ldots q] = \alpha.$$

Also, if the whole region starting $j''$ matches the pattern, then the character $\text{pat}[q-|\alpha|]$ preceding the instance of $\alpha$ ending at $q$, has to be the same as the previous iteration's 'bad-character' $\text{txt}[j+k-1]$, and therefore different from $\text{pat}[k]$:

$$\text{pat}[q-|\alpha|] = \text{txt}[j+k-1] \neq \text{pat}[k].$$

However, we are dealing with the case where $\texttt{gs}[k+1] = 0$ which leads to a contradiction, as the this means there is an instance of $\alpha$ in the pattern whose preceding character is not same as $\text{pat}[k]$ and, if true, $\texttt{gs}[k+1]$ should have stored $q > 0$, which is not the case. So the assertion is invalid.

**Case 2:** Consider the case where $q < |\alpha|$:




If shifting from $j \to j''$ yielded a full match and $q < |\alpha|$, it must be true that the characters $\text{pat}[1 \ldots q]$ will be opposite the region of text $\text{txt}[j+m-q \ldots j+m-1]$ that previously matched with some **suffix** of the good suffix $\alpha$. Why? We asserted the whole region matches after the shift from $j \to j''$ (which the matched prefix shift rule missed). If $\text{txt}[j'' \ldots j''+m-1] = \text{pat}[1 \ldots m]$ we will have to conclude from the following prefix-vs-suffix match in the pattern:

$$\underbrace{\text{pat}[1 \ldots q]}_{\text{prefix of length } q} = \text{txt}[j+m-q \ldots j+m-1] = \underbrace{\text{pat}[m-q+1 \ldots m]}_{\text{suffix of length } q}.$$

Since

$$j'' < j' \implies j+m-q < j+m-p \implies q > p.$$

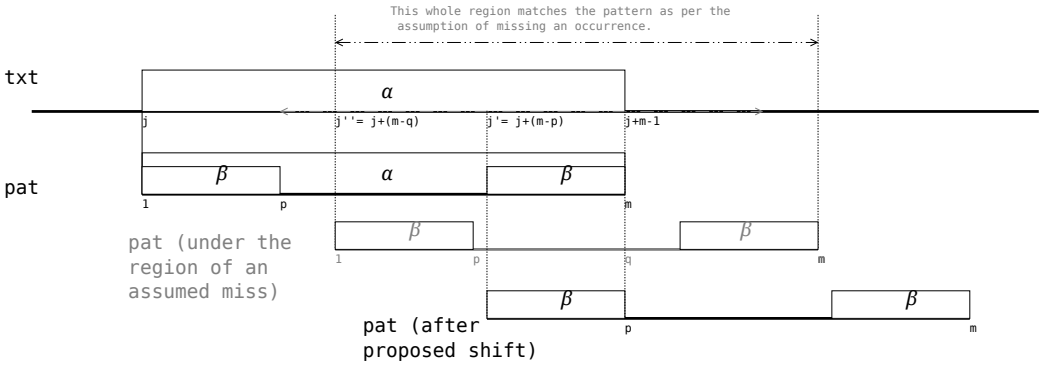
This means the suffix $\text{pat}[m-q+1 \ldots m]$ has to be longer than $|\beta|$. But, by definition, $\beta$ is the longest suffix of $\alpha$ that matches the prefix of the pattern, and now we seem to have arrived at a suffix longer than $|\beta|$ but shorter than $\alpha$ (since $q < |\alpha|$) leading to a contradiction.

Therefore the assertion that the matched prefix shift could skip unsafely over an occurrence of pattern remains invalid, and proposed matched prefix shift from $j \to j'$ is always safe.

8. Consider some iteration of the Boyer-Moore algorithm involving the pattern $\texttt{pat}[1 \ldots m]$ and the text $\texttt{txt}[1 \ldots n]$. Suppose that in this iteration the region $\texttt{txt}[j \ldots j+m-1]$ of the text is being compared with pat.

   Prove that if a right-to-left scan results in $\texttt{pat}[1 \ldots m] = \texttt{txt}[j \ldots j+m-1]$, then the shift by $m - \texttt{mp}[2]$ (in 1-based indexing, note) using the special application of matched prefix rule is **safe**, i.e., such a shift can never move pat past another such occurrence in txt.

   **Ans. 8**




Consider an iteration where $\text{pat}[1 \ldots m]$ is being compared with the region of text, $\text{txt}[j \ldots j+m-1]$, and right-to-left scan resulted in the whole region matching the pattern ($\alpha = \text{pat}[1 \ldots m]$).

The proposed shift rule of the Boyer-Moore algorithm requires us to shift by $m - p$ positions to position $j' = j + m - p$, where $p = \text{mp}[2]$ (in 1 based indexing), denotes the length of the longest proper suffix ($\beta$) of the pattern. From the propose shift from $j \to j'$ we have this identity:

$$\text{pat}[1 \ldots p] = \text{pat}[m - p + 1 \ldots m] = \text{txt}[j + m - p \ldots j + m - 1] = \beta.$$

We will prove that this shift is indeed safe using a proof by contradiction. Assume that shifting the pattern from $j \to j'$ is unsafe and skips over an occurrence of the pattern in the text starting at some $j'' = j + m - q < j'$.

In other words, the assumption is that the shift rightwards should have been, more conservatively, by some $m - q$ positions, instead of the $m - p$ positions, to have found the occurrence:

$$\text{txt}[j'' \ldots j'' + m - 1] = \text{pat}[1 \ldots m],$$

where $j < j'' < j'$.

If shifting from $j \to j''$ yielded a full match, it must be true that the characters $\text{pat}[1 \ldots q]$ will be opposite the region of text $\text{txt}[j+m-q \ldots j+m-1]$. Because $\text{txt}[j'' \ldots j''+m-1] = \text{pat}[1 \ldots m]$, we have $\text{pat}[1 \ldots q] = \text{txt}[j+m-q \ldots j+m-1]$. But in the previous iteration, $\text{txt}[j \ldots j+m-1] = \text{pat}[1 \ldots m]$, so $\text{txt}[j + m - q \ldots j + m - 1] = \text{pat}[m - q + 1 \ldots m]$. Therefore, we can conclude the following prefix-suffix relationship:

$$\text{pat}[1 \ldots q] = \text{pat}[m - q + 1 \ldots m].$$

Since

$$j'' < j' \implies j + m - q < j + m - p \implies q > p.$$

This means the suffix $\text{pat}[m - q + 1 \ldots m]$ has to be longer than $|\beta| = p$. But, by definition, $\beta$ has to be the longest proper suffix of the whole pattern that matches the prefix, leading to a contradiction.

Therefore the assertion that the proposed shift could skip unsafely over an occurrence of pattern remains invalid.

-=0=-

END

-=0=-